

Calcolabilità e complessità

Alessandro Carella

Indice

1	Automi e linguaggi	2
1.1	Automi finiti	2
1.2	Non determinismo	6
1.3	Espressioni regolari	14
1.3.1	Definizione formale di espressione regolare	14
1.3.2	Equivalenza con gli automi finiti	16
1.4	Linguaggi non regolari	22
1.4.1	Il pumping lemma per i linguaggi regolari	22
2	Linguaggi context-free	26
2.1	Grammatiche context-free	26
2.1.1	Ambiguità	27
2.1.2	Forma normale di Chomsky	28
2.1.3	Membership problem	31
2.2	Automi a pila	34
2.2.1	Definizione formale di automa a pila	35
2.2.2	Equivalenza con le grammatiche context-free	38
2.3	Linguaggi non context-free	45
2.3.1	Il pumping lemma per i linguaggi context-free	46

Capitolo 1

Automi e linguaggi

1.1 Automi finiti

Gli automi finiti sono un buon modello per computer una quantità estremamente di memoria che permette di fare molte cose utili. Il sistema di controllo per una porta automatica è un esempio di tale dispositivo.

Gli automi finiti e la loro controparte probabilistica, le *catene di Markov* sono utili strumenti quando cerchiamo di riconoscere pattern in dati. Questi dispositivi sono usati nell'elaborazione vocale e nel riconoscimento ottico dei caratteri. Le catene di Markov sono state perfino usate per modellare e predire i cambiamenti dei prezzi nei mercati finanziari.

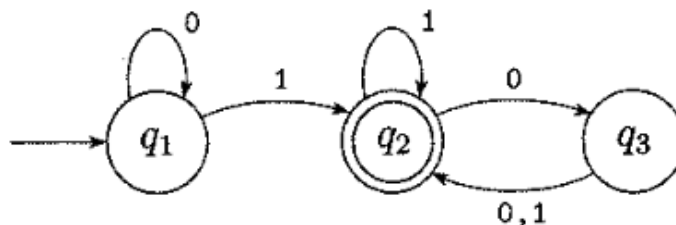
Definizione formale di automa finito

DEFINIZIONE 1.1: AUTOMA FINITO

Un *automa finito* è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove:

1. Q è un insieme finito chiamato l'insieme degli *stati*;
2. Σ è un insieme finito chiamato *alfabeto*;
3. $\delta : Q \times \Sigma \rightarrow Q$ è la *funzione di transizione*;
4. $q_0 \in Q$ è lo *stato iniziale*;
5. $F \subseteq Q$ è l'*insieme degli stati accettanti*.

Consideriamo il seguente automa M_1 :



Possiamo descrivere M_1 formalmente ponendo $M_1 = (Q, \Sigma, \delta, q_1, F)$ ove:

1. $Q = \{q_1, q_2, q_3\}$;
2. $\Sigma = \{0, 1\}$;
3. δ è descritta come segue:

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

4. q_1 è lo stato iniziale;
5. $F = \{q_2\}$

Se A è l'insieme di tutte le stringhe che la macchina M accetta, diciamo che A è il *linguaggio della macchina* M e scriviamo $L(M) = A$. Diciamo che M *riconosce* A o che M *accetta* A . Poichè il termine *accetta* ha significati diversi quando ci riferiamo a macchine che accettano stringhe e macchine che accettano linguaggi, per evitare confusione preferiamo per i linguaggi il termine *riconosce*.

Una macchina può accettare diverse stringhe, ma riconosce sempre soltanto un linguaggio. Se la macchina non accetta alcuna stringa, essa riconosce ancora un linguaggio, il linguaggio vuoto $\emptyset$.

Nel nostro esempio, sia:

$$A = \{w \mid w \text{ contiene almeno un } 1 \text{ e un numero pari di } 0 \text{ segue l'ultimo } 1\}$$

Definizione formale di computazione

Finora abbiamo descritto gli automi finiti informalmente, usando diagrammi di stato, e con una definizione formale, come una quintupla. La descrizione informale è più facile da comprendere all'inizio, ma la definizione formale è più utile per rendere la notazione precisa, risolvendo ogni ambiguità.

Facciamo lo stesso per la computazione di un automa finito.

Sia $M = (Q, \Sigma, \delta, q_0, F)$ un automa finito e sia $w = w_1 w_2 \cdots w_n$ una stringa dove ogni w_i è un elemento dell'alfabeto Σ . Allora M accetta w se esiste una sequenza di stati $r_0, r_1, \dots, r_n$ in Q con tre condizioni:

1. $r_0 = q_0$;
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ per $i = 0, \dots, n-1$;
3. $r_n \in F$.

La prima condizione afferma che la macchina inizia nello stato iniziale. La seconda condizione afferma che la macchina passa da stato a stato in base alla funzione di transizione. La terza condizione afferma che la macchina accetta il suo input se termina la lettura in uno stato accettante. Diciamo che M riconosce il linguaggio A se $A = \{w \mid M \text{ accetta } w\}$.

DEFINIZIONE 1.2: LINGUAGGIO REGOLARE

Un linguaggio è chiamato un *linguaggio regolare* se un automa finito lo riconosce.

Le operazioni regolari

Dopo aver introdotto e definito gli automi finiti e i linguaggi regolari, iniziamo a studiarne le proprietà. In aritmetica, gli oggetti di base sono i numeri e gli strumenti sono le operazioni per trattarli, come $+$ e $\times$. Nella teoria della computazione, gli oggetti sono i linguaggi e gli strumenti includono operazioni specificatamente progettate per trattarli. Definiamo tre operazioni sui linguaggi, chiamate *operazioni regolari*, e le usiamo per studiare le proprietà dei linguaggi regolari.

DEFINIZIONE 1.3: OPERAZIONI REGOLARI

Siano A e B linguaggi. Definiamo le operazioni regolari *unione*, *concatenazione* e *star* come segue:

- **Unione:** $A \cup B = \{x \mid x \in A \text{ o } x \in B\}$
- **Concatenazione:** $A \circ B = \{xy \mid x \in A \text{ e } y \in B\}$

- **Star:** $A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ e ogni } x_i \in A\}$

Esempio

Sia Σ l'alfabeto standard a 26 lettere $\{a, b, \dots, z\}$. Se $A = \{good, bad\}$ e $B = \{boy, girl\}$, allora:

$$A \cup B = \{good, bad, boy, girl\}$$

$$A \circ B = \{goodboy, goodgirl, badboy, badgirl\}$$

$$A^* = \{\epsilon, good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, \dots\}$$

Sia $\mathbb{N} = \{1, 2, 3, \dots\}$ l'insieme dei numeri naturali. Quando diciamo che $\mathbb{N}$ è *chiuso rispetto alla moltiplicazione*, intendiamo che per ogni x e per ogni y in $\mathbb{N}$, il prodotto $x \times y \in \mathbb{N}$. Invece $\mathbb{N}$ non è chiuso rispetto alla divisione, poichè 1 e 2 sono in $\mathbb{N}$ ma $1/2 \notin \mathbb{N}$.

In generale, una classe di oggetti è *chiusa* rispetto a un'operazione se l'applicazione di questa operazioni a elementi della classe restituisce un oggetto ancora nella classe. Mostriamo che la classe dei linguaggi regolari è chiusa rispetto a tutte e tre le operazioni regolari.

Teorema 1.1: Chiusura unione

La classe dei linguaggi regolari è chiusa rispetto all'operazioni di unione.

In altre parole, se A_1 e A_2 sono linguaggi regolari, lo è anche $A_1 \cup A_2$.

Quindi, abbiamo due linguaggi regolari, A_1 e A_2 , e vogliamo mostrare che anche $A_1 \cup A_2$ è regolare. Poichè A_1 e A_2 sono regolari, sappiamo che sono riconosciuti da degli automi finiti. Quindi per provare che $A_1 \cup A_2$ è regolare, dobbiamo esibire un automa finito, M , che riconosce $A_1 \cup A_2$.

Dimostrazione. Supponiamo che M_1 riconosca A_1 , dove $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ ed M_2 riconosca A_2 , dove $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$. Costruiamo M che riconosce $A_1 \cup A_2$ dove $M = (Q, \Sigma, \delta, q_0, F)$.

1. $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ e } r_2 \in Q_2\}$.
Questo insieme è il *prodotto cartesiano* degli insiemi Q_1 e Q_2 ed è denotato con $Q_1 \times Q_2$.
2. Σ , l'alfabeto è lo stesso di M_1 e M_2 . In questo teorema e in quelli successivi, assumiamo per semplicità che sia M_1 che M_2 abbiano stesso alfabeto di simboli. Tuttavia, se così non fosse, $\Sigma = \Sigma_1 \cup \Sigma_2$.

3. δ , la funzione di transizione, è definita come segue. Per ogni $(r_1, r_2) \in Q$ e ogni $a \in \Sigma$, sia

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

4. q_0 è la coppia (q_1, q_2) .
5. F è l'insieme delle coppie in cui l'uno o l'altro elemento è uno stato accettante di M_1 o M_2 . Possiamo scriverlo come

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ o } r_2 \in F_2\}$$

Questa espressione è uguale a $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

□

Abbiamo mostrato che l'unione di due linguaggi regolari è regolare, provando che la classe dei linguaggi regolari è chiusa rispetto all'operazione di unione. Ora mostriamo che la classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

Teorema 1.2: Chiusura concatenazione

La classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

In altre parole, se A_1 e A_2 sono linguaggi regolari, allora lo è anche $A_1 \circ A_2$.

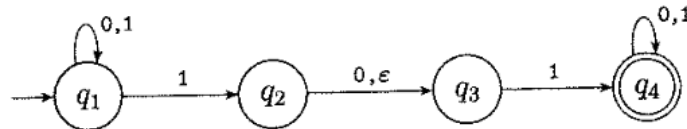
Per provare questo teorema, possiamo iniziare con gli automi finiti M_1 ed M_2 che riconoscono i linguaggi regolari A_1 ed A_2 . Ma ora, invece di costruire M che accetta l'input se M_1 o M_2 lo accetta, esso deve accettare se il suo input può essere diviso in due parti, tali che M_1 accetta la prima parte ed M_2 accetta la seconda parte. Il problema è che M non sa dividere il suo input. Per risolvere questo problema, introduciamo una tecnica chiamata *non determinismo*.

1.2 Non determinismo

Negli esempi visti precedentemente, ogni passo di una computazione seguiva univocamente dal passo precedente. Quando la macchina è in un dato stato e legge il simbolo di input successivo, sappiamo qual è lo stato successivo, esso è univocamente determinato. In questo caso parliamo di computazione *deterministica*. In una macchina *non deterministica*, possono esistere diverse scelte per lo

stato successivo in ogni punto.

Il non determinismo è una generalizzazione del determinismo, quindi ogni automa finito deterministico è automaticamente un automa finito non deterministico. Consideriamo il seguente automa:



La differenza tra un automa finito deterministico, DFA, e un automa finito non deterministico, indicato con NFA, è evidente. In primo luogo, ogni stato di un DFA ha sempre esattamente un arco uscente per ogni simbolo nell'alfabeto. L'NFA viola questa regola, infatti, lo stato q_1 dell'automata della figura precedente, ha un arco uscente per 0, ma ne ha due per 1. In un NFA, uno stato può avere zero, uno, o più archi uscenti per ogni simbolo dell'alfabeto.

Inoltre, in un DFA, le etichette sugli archi di transizione sono simboli dell'alfabeto. Questo NFA ha un arco con etichetta ϵ .

Durante la computazione di un NFA, quando si giunge ad uno stato con più modi per procedere, la macchina si divide in più copie di sè stessa e segue tutte le possibilità in parallelo. Infine, se una qualunque di queste copie della macchina è in uno stato accettante alla fine dell'input, l'NFA accetta la stringa di input.

Il non determinismo può essere visto come una specie di computazione parallela dove "processi" multipli indipendenti o "threads" possono essere eseguiti contemporaneamente. Un altro modo per considerare una computazione non deterministica è vederla come un albero di possibilità. La radice dell'albero corrisponde all'inizio della computazione. Considerando sempre l'automata precedente, l'albero di computazione è il seguente:

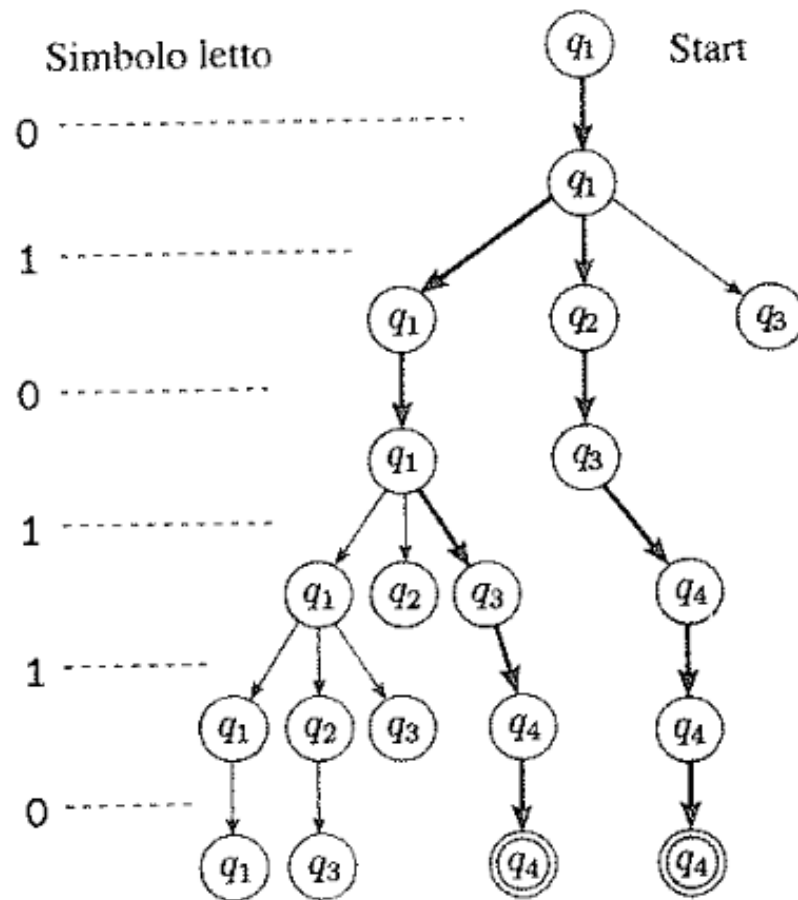


Figura 1.1: La computazione dell'automa precedente su 010110

Definizione formale di automa finito non deterministico

La definizione formale di automa finito non deterministico è simile a quella di automa finito deterministico. Essi differiscono nel tipo di funzione di transizione. In un DFA, la funzione di transizione prende uno stato e un simbolo di input e produce lo stato successivo. In un NFA, la funzione di transizione prende uno stato e un simbolo di input o *la stringa vuota* e produce l'*insieme dei possibili stati successivi*.

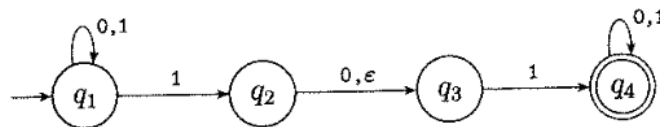
DEFINIZIONE 1.4: AUTOMA FINITO NON DETERMINISTICO

Un *automa finito non deterministico* è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove

1. Q è un insieme finito di stati,
2. Σ è un alfabeto finito,
3. $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ è la funzione di transizione,
4. $q_0 \in Q$ è lo stato iniziale,
5. $F \subseteq Q$ è l'insieme degli stati di accettazione.

Esempio

Riconsideriamo il seguente NFA N_1 :



La descrizione formale di N_1 è $(Q, \Sigma, \delta, q_1, F)$ dove

1. $Q = \{q_1, q_2, q_3, q_4\}$
2. $\Sigma = \{0, 1\}$
3. δ è data da

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

4. q_1 è lo stato iniziale
5. $F = \{q_4\}$

La definizione formale di computazione per un NFA è simile a quella per un DFA. Sia $N = (Q, \Sigma, \delta, q_0, F)$ un NFA e sia w una stringa sull'alfabeto Σ . Diciamo

che N accetta w se possiamo scrivere w come $w = y_1 y_2 \dots y_m$, dove ciascun y_i è un elemento di $\sum_\epsilon$ ed esiste una sequenza di stati $r_0, r_1, \dots, r_n$ in Q con tre condizioni:

1. $r_0 = q_0$
2. $r_{i+1} \in \delta(r_i, y_{i+1})$ per $i = 0, \dots, m-1$
3. $r_m \in F$

Equivalenza tra gli NFA e i DFA Gli automi finiti deterministici e quelli non deterministici riconoscono la stessa classe di linguaggi. Diciamo che due macchine sono *equivalenti* se esse riconoscono lo stesso linguaggio.

Teorema 1.3: Equivalenza NFA e DFA

Per ogni automa finito non deterministico esiste un automa finito deterministico equivalente.

Se un linguaggio è riconosciuto da un NFA, allora dobbiamo mostrare l'esistenza di un DFA che lo riconosce. L'idea è trasformare l'NFA in un DFA equivalente che simula l'NFA.

Dimostrazione. Sia $N = (Q, \sum, \delta, q_0, F)$ l'NFA che riconosce un linguaggio A . Costruiamo un DFA $M = (Q', \sum, \delta', q'_0, F')$ che riconosce A . Prima di fare la costruzione completa, consideriamo inizialmente il caso più semplice in cui N non ha ϵ -archi.

1. $Q' = \mathcal{P}(Q)$
Ogni stato di M è un insieme di stati di N .
2. Per $R \in Q'$ e $a \in \sum$, sia $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a)\}$ per qualche $r \in R$
Se R è uno stato di M , esso è anche un insieme di stati di N . Quando M legge un simbolo a nello stato R . Poichè da ogni stato si può andare in un insieme di stati, prendiamo l'unione di tutti questi insiemi. Un altro modo per scrivere quest'espressione è:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

3. $q'_0 = \{q_0\}$

4. $F' = \{R \in Q' \mid R \text{ contiene uno stato accettante di } N\}$

Ora dobbiamo considerare gli ϵ -archi. Per farlo introduciamo qualche ulteriore notazione. Per ogni stato R di M , definiamo $E(R)$ come la collezione di stati che possono essere raggiunti dagli elementi di R proseguendo solo con ϵ -archi, includendo gli stessi elementi di R . Formalmente, per $R \subseteq Q$ sia

$$E(R) = \{q \mid q \text{ può essere raggiunto da } R \text{ attraverso 0 o più } \epsilon\text{-archi}\}$$

Modifichiamo la funzione di transizione di M ponendo le dita su tutti gli stati che possono essere raggiunti proseguendo attraverso ϵ -archi dopo ogni passo. Sostituendo $\delta(r, a)$ con $E(\delta(r, a))$ realizziamo questo effetto. La funzione di transizione è la seguente.

$$\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a)) \text{ per qualche } r \in R\}$$

Inoltre, dobbiamo modificare lo stato iniziale di M per muovere inizialmente le dita su tutti i possibili stati che possono essere raggiunti dallo stato iniziale di N attraverso gli ϵ -archi. Cambiare q'_0 in $E(\{q_0\})$ realizza questo risultato. Abbiamo così completato la costruzione del DFA M che simula l'NFA N . $\square$

Corollario 1.1

Un linguaggio è regolare se e solo se qualche automa finito non deterministico lo riconosce.

Una direzione della condizione “se e solo se” afferma che un linguaggio è regolare se qualche NFA lo riconosce. Tuttavia, abbiamo dimostrato che ogni NFA può essere trasformato in un DFA. Conseguentemente, se un NFA riconosce un linguaggio, lo fa anche qualche DFA, e quindi il linguaggio regolare. L'altra direzione della condizione “se e solo se” afferma che un linguaggio è regolare solo se qualche NFA lo riconosce. Questa condizione è vera perchè un linguaggio regolare ha un DFA che lo riconosce e un DFA è anche un NFA.

Chiusura rispetto alle operazioni regolari

Torniamo alla chiusura della classe dei linguaggi regolari rispetto alle operazioni regolari. Dobbiamo provare che l'unione, la concatenazione e lo star di linguaggi regolari sono ancora regolari.

Teorema 1.4

La classe dei linguaggi regolari è chiusa rispetto all'operazione di unione.

Abbiamo i linguaggi regolari A_1 ed A_2 e vogliamo provare che $A_1 \cup A_2$ è regolare. L'idea è provare che due NFA, N_1 ed N_2 per A_1 ed A_2 , e comporli in un nuovo NFA, N . La macchina N deve accettare il suo input se N_1 o N_2 accetta questo input. La nuova macchina ha uno stato iniziale che si dirama negli stati iniziali delle vecchie macchine con ϵ archi.

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ per riconoscere $A_1 \cup A_2$.

1. $Q = \{q_0\} \cup Q_1 \cup Q_2$
2. Lo stato q_0 è lo stato iniziale di N
3. L'insieme degli stati accettanti è $F = F_1 \cup F_2$
4. Definiamo δ in modo che per ogni $q \in Q$ e $a \in \Sigma_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$

□

Teorema 1.5: L

classe dei linguaggi regolari è chiusa rispetto all'operazione di concatenazione.

L'idea è prendere due NFA, N_1 ed N_2 , per A_1 ed A_2 , e combinarli in un nuovo NFA N . Poniamo come stato iniziale di N lo stato iniziale N_1 . Gli stati accettanti di N_1 hanno degli ulteriori ϵ -archi che non deterministicamente permettono di diramarsi in N_2 ogni volta che N_1 è in uno stato accettante, indicando che ha trovato un pezzo iniziale dell'input che costituisce una stringa in A_1 . Gli stati accettandi di N sono gli stati accettanti di N_2 .

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2 .

Costruiamo $N = (Q, \Sigma, \delta, q_1, F_2)$ per riconoscere $A_1 \circ A_2$.

1. $Q = Q_1 \cup Q_2$

2. Lo stato q_1 è uguale allo stato iniziale di N_1
3. Gli stati accettanti F_2 sono uguale agli stati accettanti di N_2
4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \text{ e } a = \epsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

□

Teorema 1.6

La classe dei linguaggi regolari è chiusa rispetto all'operazione star.

Prendiamo un NFA N_1 per A_1 e lo modifichiamo per riconoscere A_1^* . L'NFA N risultante accetterà il suo input quando esso può essere diviso in varie parti ed N_1 accetta ogni parte. Possiamo costruire N come N_1 con ϵ -archi supplementari che dagli stati accettanti ritornano allo stato iniziale. In questo modo, quando l'elaborazione giunge alla fine di una parte che N_1 accetta, la macchina N ha la scelta di tornare indietro allo stato iniziale per provare a leggere un'altra parte che N_1 accetta. Inoltre, dobbiamo modificare N in modo che accetti ϵ , che è sempre un elemento di A_1^* .

Dimostrazione. Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 . Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ per riconoscere A_1^* .

1. $Q = \{q_0\} \cup Q_1$
2. Lo stato q_0 è il nuovo stato iniziale.
3. $F = \{q_0\} \cup F_1$
4. Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\epsilon$

$$\delta(q, a) = \begin{cases} \delta_1(qa) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q, a) & q \in F_1 \text{ e } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ e } a = \epsilon \\ \{q_1\} & q = q_0 \text{ e } a = \epsilon \\ \emptyset & q = q_0 \text{ e } a \neq \epsilon \end{cases}$$



1.3 Espressioni regolari

Possiamo usare le operazioni regolari per costruire espressioni che descrivono linguaggi, che sono chiamate *espressioni regolari*. Ad esempio:

$$(0 \cup 1)0^*$$

Il valore di un'espressione regolare è un linguaggio. In questo caso, il valore è il linguaggio che consiste di tutte le stringhe che iniziano con 0 o un 1 seguito da un qualsiasi numero di simboli uguali a 0.

1.3.1 Definizione formale di espressione regolare

DEFINIZIONE 1.5: ESPRESSIONE REGOLARE

Diciamo che R è un'espressione regolare se R è

1. a per qualche a nell'alfabeto Σ ;
2. ϵ ;
3. $\emptyset$;
4. $(R_1 \cup R_2)$ dove R_1 ed R_2 sono espressioni regolari;
5. $(R_1 \circ R_2)$ dove R_1 ed R_2 sono espressioni regolari;
6. $(R_1)^*$ dove R_1 è un'espressione regolare.

È bene non confondere le espressioni regolari ϵ e $\emptyset$. L'espressione regolare ϵ rappresenta il linguaggio che contiene la sola stringa vuota, mentre $\emptyset$ rappresenta il linguaggio che non contiene alcuna stringa.

Inoltre, per comodità, usiamo R^+ come abbreviazione per RR^* e possiamo scrivere R^* come $R^+ \cup \epsilon$.

Quando vogliamo distinguere un'espressione regolare R e il linguaggio che descrive, denotiamo con $L(R)$ il linguaggio di R .

Analizziamo alcune proprietà.

1. $a^* \emptyset = \emptyset$
Concatenando l'insieme vuoto a un qualsiasi insieme si ottiene l'insieme vuoto.
2. $\emptyset^* = \{\epsilon\}$
L'operazione star concatena un numero qualsiasi di stringhe del linguaggio per ottenere una stringa nel risultato. Se il linguaggio è vuoto, l'operazione star può concatenare 0 stringhe, dando solo la stringa vuota.
3. $R \cup \emptyset = R$
Aggiungere il linguaggio vuoto a un qualsiasi altro linguaggio non lo cambia.
4. $R \circ \epsilon = R$
Concatenare la stringa vuota a una qualsiasi stringa non la cambia.

Tuttavia, scambiare $\emptyset$ e ϵ nelle precedenti identità può far sì che le uguaglianze non siano vere.

1. $R \cup \epsilon$ può non essere uguale a R
Per esempio, se $R = \emptyset$, allora $L(R) = \{\emptyset\}$ ma $L(R \cup \epsilon) = \{\emptyset, \epsilon\}$.
2. $R \circ \emptyset$ può non essere uguale a R .
Per esempio, se $R = \emptyset$, allora $L(R) = \{\emptyset\}$ ma $L(R \circ \emptyset) = \emptyset$

Le espressioni regolari sono un utile strumento nel progetto dei compilatori per i linguaggi di programmazione. Gli oggetti elementari in un linguaggio di programmazione, chiamati *token*, come i nomi delle variabili e le costanti, possono essere descritti con espressioni regolari. Ad esempio, una costante numerica può avere una parte decimale e/o un segno e può essere descritta come un elemento del linguaggio

$$(+ \cup - \cup \epsilon)(D^+ \cup D^+.D^* \cup D^*.D^+)$$

Una volta che la sintassi di un linguaggio di programmazione è stata descritta con un'espressione regolare in termini dei suoi token, dei sistemi automatici possono generare l'*analizzatore lessicale*, la parte di un compilatore che elabora inizialmente il programma input.

1.3.2 Equivalenza con gli automi finiti

Le espressioni regolari e gli automi finiti sono equivalenti rispetto alla loro potenza descrittiva. Ogni espressione regolare può essere trasformata in un automa finito che riconosce il linguaggio che essa descrive, e viceversa.

Teorema 1.7

Un linguaggio è regolare se e solo se qualche espressione regolare lo descrive.

Questo teorema deve essere dimostrato in entrambe le direzioni. Enunciamo e proviamo ciascuna direzione in un lemma separato.

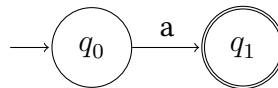
LEMMA 1.1

Se un linguaggio è descritto da un'espressione regolare, allora esso è regolare.

Supponiamo di avere un'espressione regolare R che descrive un linguaggio A . Mostriamo come trasformare R in un NFA che riconosce A .

Dimostrazione. Trasformiamo R in un NFA N . Consideriamo i sei casi nella definizione di **espressione regolare**.

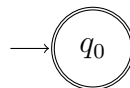
1. $R = a$ per qualche $a \in \Sigma$. Allora $L(R) = \{a\}$ e il seguente NFA riconosce $L(R)$.



Questa macchina soddisfa la definizione di NFA ma non quella di DFA perchè ha qualche stato con nessun arco uscente per ogni possibile simbolo di input.

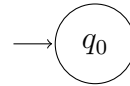
Formalmente, $N = (\{q_0, q_1\}, \Sigma, \sigma, q_0, \{q_1\})$ dove descriviamo σ dicendo che $\sigma(q_0, a) = \{q_1\}$ e $\sigma(r, b) = \emptyset$ per $r \neq q_0$ o $b \neq a$.

2. $R = \epsilon$. Allora $L(R) = \{\epsilon\}$ e il seguente NFA riconosce $L(R)$.



Formalmente $N = (\{q_0\}, \Sigma, \sigma, q_0, \{q_0\})$, dove $\sigma(r, b) = \emptyset$ per ogni r e b .

3. $R = \emptyset$. Allora $L(R) = \emptyset$ e il seguente NFA riconosce $L(R)$.



Formalmente, $N = (\{q_0\}, \Sigma, \sigma, q_0, \emptyset)$ dove $\sigma(r, b) = \emptyset$ per ogni r e b .

4. $R = R_1 \cup R_2$
 5. $R = R_1 \circ R_2$
 6. $R = R_1^*$

Per gli ultimi tre casi, usiamo le costruzioni date nelle prove che la classe dei linguaggi regolari è chiusa rispetto alle operazioni regolari. $\square$

Questo termina la prima parte del teorema dando la direzione più facile della condizione “se e solo se”.

LEMMA 1.2

Se un linguaggio è regolare, allora è descritto da un'espressione regolare.

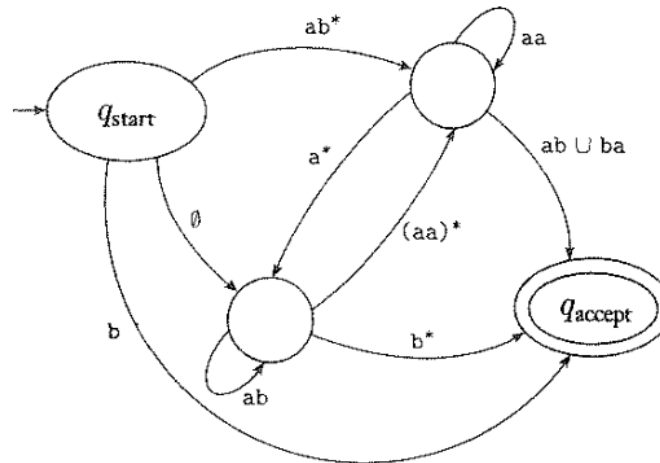
Dobbiamo mostrare che se un linguaggio A è regolare, allora un'espressione regolare lo descrive. Poiché A è regolare, esso è accettato da un DFA. Descriviamo una procedura per trasformare i DFA in espressioni regolari equivalenti.

Dividiamo questa procedura in due parti, usando un nuovo tipo di automa finito chiamato *automa finito non deterministico generalizzato*, GNFA.

Prima mostriamo come trasformare un DFA in un GNFA e poi come trasformare un GNFA in un'espressione regolare.

Gli automi finiti non deterministici generalizzati sono automi finiti non deterministici nei quali gli archi delle transizioni possono avere espressioni regolari come etichette, invece che solo elementi dell'alfabeto o ϵ . Un GNFA è non deterministico e quindi può avere diversi modi di elaborare la stessa stringa di input.

La figura seguente presenta un esempio di GNFA.



Per comodità, richiediamo che i GNFA abbiano sempre una forma speciale che soddisfi le seguenti condizioni.

- Lo stato iniziale ha archi di transizione uscenti verso un qualsiasi altro stato ma nessun arco entrante proveniente da un qualsiasi altro stato.
- Esiste un solo stato accettante, ed esso ha archi entranti provenienti da un qualsiasi altro stato ma nessun arco uscente verso un qualsiasi altro stato. Inoltre, lo stato accettante non è uguale allo stato iniziale.
- Eccetto che per lo stato iniziale e lo stato accettante, un arco va da ogni stato ad ogni altro stato e anche da ogni stato a sé stesso.

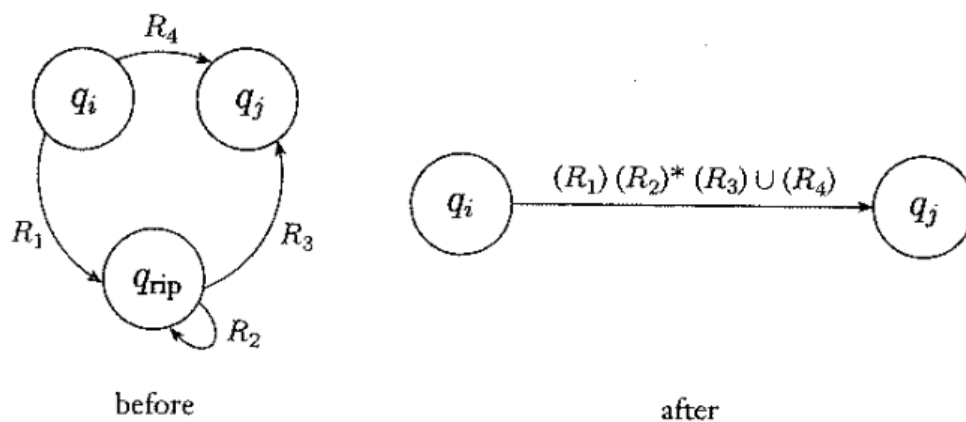
Possiamo facilmente trasformare un DFA in un GNFA. Aggiungiamo semplicemente un nuovo stato iniziale con un ϵ -arco che entra nel vecchio stato iniziale e un nuovo stato accettante con ϵ -archi entranti, provenienti dai vecchi stati accettanti. Se alcuni archi hanno più etichette (o se ci sono più archi che collegano gli stessi due stati nella stessa direzione), sostituiamo ognuno di essi con un solo arco la cui etichetta è l'unione delle precedenti etichette. Infine, aggiungiamo archi con etichetta $\emptyset$ tra stati che non hanno archi. Quest'ultimo non cambierebbe il linguaggio riconosciuto perchè una transizione etichettata con $\emptyset$ non può mai essere usata.

Ora mostriamo come trasformare un GNFA in un'espressione regolare. Supponiamo che il GNFA abbia k stati. Allora, poichè un GNFA deve avere uno stato

iniziale e uno stato accettante ed essi devono essere diversi tra loro, sappiamo che $k \geq 2$. Se $k > 2$, costruiamo un GNFA equivalente con $k - 1$ stati. Questo passo può essere ripetuto sul nuovo GNFA fino a quando esso è ridotto a 2 stati. Se $k = 2$, il GNFA ha un solo arco che va dallo stato iniziale allo stato accettante. L'etichetta di questo arco è l'espressione regolare equivalente.

Il passo cruciale è costruire un GNFA equivalente con uno stato in meno quando $k > 2$. Lo facciamo scegliendo uno stato, estraendolo dalla macchina, e modificando il resto in modo che sia riconosciuto lo stesso linguaggio. Ogni stato può essere scelto, purché non sia lo stato iniziale o lo stato accettante. Chiamiamo q_{rip} lo stato rimosso.

Dopo aver rimosso q_{rip} modifichiamo la macchina cambiando le espressioni regolari che etichettano ciascuno dei restanti archi. Le nuove etichette controbilanciano l'assenza di q_{rip} reintegrando le computazioni perse. La nuova etichetta che va da uno stato q_i a uno stato q_j è un'espressione regolare che descrive tutte le stringhe che porterebbero la macchina da q_i a q_j o direttamente o tramite q_{rip} . Analizziamo tale procedimento nella seguente figura.



Nella vecchia macchina, se

1. q_i va a q_{rip} con un arco etichettato R_1 ;
2. q_{rip} va in sé stesso con un arco etichettato R_2 ;
3. q_{rip} va a q_j con un arco etichettato R_3 ;
4. q_i va a q_j con un arco etichettato R_4 ,

allora nella nuova macchina, l'arco da q_i a q_j riceve l'etichetta

$$(R_1)(R_2)^*(R_3) \cup (R_4)$$

Facciamo questa modifica per ogni arco che va da un qualsiasi stato q_i a un qualsiasi stato q_j , includendo il caso in cui $q_i = q_j$. La nuova macchina riconosce il linguaggio di partenza.

Dimostrazione. Formalizziamo quest'idea. In primo luogo, definiamo formalmente l'automa GNFA. Un GNFA è simile ad un automa finito non deterministico tranne che per la funzione di transizione, che ha la forma

$$\delta : (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \rightarrow R$$

Il simbolo R è la collezione di tutte le espressioni regolari sull'alfabeto Σ e q_{start} e q_{accept} sono gli stati iniziale e accettante.

Il dominio della funzione di transizione è $(Q - \{q_{accept}\}) \times (Q - \{q_{start}\})$ perchè un arco collega ogni stato a un qualsiasi altro stato, tranne che nessun arco è uscente da q_{accept} o è entrante in q_{start} . $\square$

DEFINIZIONE 1.6: AUTOMA FINITO NON DETERMINISTICO GENERALIZZATO

Un *automa finito non deterministico generalizzato* è una quintupla $(Q, \Sigma, \delta, q_{start}, q_{accept})$, dove

1. Q è l'insieme finito degli stati;
2. Σ è l'alfabeto di input;
3. $\delta : (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \rightarrow R$ è la funzione di transizione;
4. q_{start} è lo stato iniziale;
5. q_{accept} è lo stato accettante.

Un GNFA accetta una stringa w in Σ^* se $w = w_1 w_2 \cdots w_k$, dove ogni w_i è in Σ^* ed esiste una sequenza di stati $q_0, q_1, \dots, q_k$ tale che

1. $q_0 = q_{start}$ è lo stato iniziale,
2. $q_k = q_{accept}$ è lo stato accettante e
3. per ogni i , risulta $w_i \in L(R_i)$, dove $R_i = \delta(q_{i-1}, q_i)$

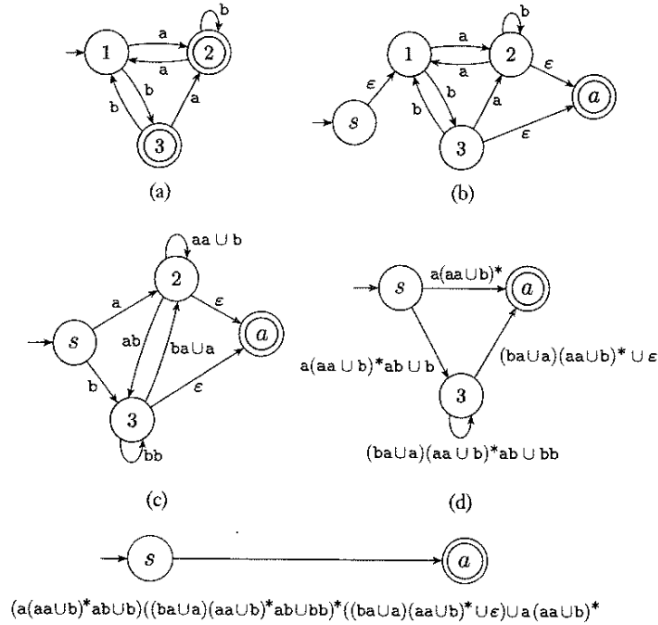
Torniamo alla dimostrazione del **lemma 1.2**, sia M il DFA per il linguaggio A . Allora trasformiamo M in un GNFA G aggiungendo un nuovo stato iniziale un nuovo stato accettante e i necessari archi di transizione supplementari. Usiamo la procedura $\text{CONVERT}(G)$, che prende un GNFA e restituisce un'espressione regolare equivalente. Questa procedura usa la *ricorsione*.

```

1 CONVERT(G) :
2   if k = 2 then return R;
3   if k > 2 then scelto  $q_{rip} \neq q_{accept}$  e  $q_{start}$ 
4     CONVERT(G')
5    $G' = (Q', \Sigma, \delta, q_{start}, q_{accept})$ 
6    $Q' = Q - \{q_{rip}\}$ 
7   foreach  $q_i \in Q' - \{q_{accept}\}$  and  $q_j \in Q' - \{q_{start}\}$ 
8      $\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4)$ 
9      $R_1 = \delta(q_i, q_{rip})$ 
10     $R_2 = \delta(q_{rip}, q_{rip})$ 
11     $R_3 = \delta(q_{rip}, q_j)$ 
12     $R_4 = \delta(q_i, q_j)$ 
13   endfor

```

Analizziamo un esempio, iniziamo con un DFA con tre stati. I passi nella trasformazione sono mostrati nella figura seguente.



1.4 Linguaggi non regolari

In questa sezione, mostriamo come provare che alcuni linguaggi non possono essere riconosciuti da alcun automa finito. Consideriamo il linguaggio $B = \{0^n 1^n \mid n \geq 0\}$. Se cerchiamo di trovare un DFA che riconosca B , scopriamo che la macchina sembra aver bisogno di ricordare quanti simboli uguali a 0 ha visto fin quando legge l'input. Poichè il numero di simboli uguali a 0 non è limitato, la macchina dovrà tenere traccia di un numero infinito di possibilità. Ma non può farlo con un numero finito di stati.

Non è detto che se un linguaggio sembra richiedere memoria non limitata allora è un linguaggio non regolare. Per esempio, consideriamo i due linguaggi sull'alfabeto $\Sigma = \{0, 1\}$:

- $C = \{w \mid w \text{ ha lo stesso numero di simboli uguali a 0 e simboli uguali a 1}\}$
- $D = \{w \mid w \text{ ha un numero uguale di occorrenze di 01 e 10 come sottostringhe}\}$

A prima vista, una macchina che riconosce sembra aver bisogno di contare in ciascun caso,, e quindi nessuno dei due linguaggi sembra essere regolare.

Sappiamo che C non è regolare, ma D è regolare.

1.4.1 Il pumping lemma per i linguaggi regolari

La tecnica per provare la regolarità deriva da un teorema sui linguaggi regolari, chiamato il *pumping lemma*. Questo teorema afferma che tutti i linguaggi regolari hanno una proprietà speciale. Se noi possiamo mostrare che un linguaggio non ha questa proprietà, siamo sicuri che esso non è regolare. La proprietà afferma che tutte le stringhe nel linguaggio possono essere “replicate” se la loro lunghezza raggiunge almeno uno specifico valore speciale, chiamato la *lunghezza del pumping*.

Teorema 1.8: Pumping lemma per linguaggi regolari

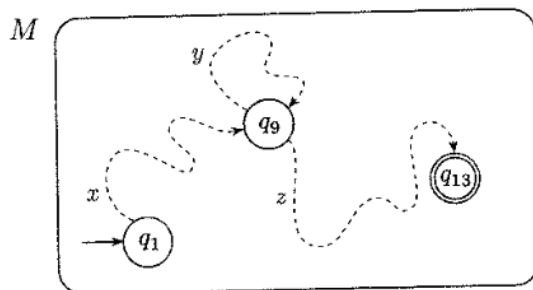
Se A è un linguaggio regolare, allora esiste p (lunghezza del pumping) tale che se s è una qualsiasi stringa in A di lunghezza almeno p , allora s può essere divisa in tre parti, $s = xyz$, soddisfacenti le seguenti condizioni:

1. $\forall i \geq 0, xy^i z \in A$
2. $|y| > 0$
3. $|xy| \leq p$

Sia $M = (Q, \Sigma, \delta, q_1, F)$ un DFA che riconosce A . Assegniamo alla lunghezza del pumping p il numero degli stati di M . Mostriamo che per ogni stringa s in A di lunghezza almeno p può essere divisa nelle tre parti xyz , soddisfacenti le tre condizioni.

Se $s \in A$ ha lunghezza almeno p , consideriamo la sequenza di stati che M attraversa nella computazione con input s . Inizia con lo stato iniziale q_1 , poi va per esempio in q_3 , poi in q_{20} , e così via, finché raggiunge la fine di s nello stato q_{13} . Se s è in A , sappiamo che M accetta s , quindi q_{13} è uno stato accettante. Se supponiamo che n sia la lunghezza di s , la sequenza di stati $q_1 q_3 q_{20} \dots q_{13}$ ha lunghezza $n + 1$. Poiché n è almeno p , sappiamo che $n + 1 > p$, il numero di stati di M . Quindi, la sequenza deve contenere uno stato che si ripete. Questo risultato è un esempio del *principio della piccionaia*.

Di seguito viene mostrato un esempio di un DFA con uno stato che si ripete.



Ora dividiamo s nelle tre componenti x , y e z . La componente x è la parte di s che compare prima di q_9 , la componente y è la parte tra le due occorrenze di q_9 e la componente z è la parte restante di s , che viene dopo la seconda occorrenza di q_9 .

Vediamo perché questa suddivisione di s soddisfa le tre condizioni. Supponiamo di eseguire M sull'input $xyyz$. Sappiamo che x porta M da q_1 a q_9 , e poi il primo y lo riporta da q_9 a q_9 , come fa il secondo y e poi z lo porta in q_{13} che accetta $xyyz$ in quanto stato accettante. Analogamente, esso accetterà $xy^i z \forall i > 0$.

Per verificare la condizione 2, vediamo che $|y| > 0$, poiché era la parte di s tra due diverse occorrenze dello stato q_9 . Per ottenere la condizione 3, ci assicuriamo che q_9 sia la prima ripetizione nella sequenza. Per il principio della piccionaia, i primi $p + 1$ stati nella sequenza devono contenere una ripetizione. Quindi $|xy| \leq p$.

Dimostrazione. Sia $M = (Q, \Sigma, \delta, q_1, F)$ un DFA che riconosce A e sia p il numero di stati di M .

Sia $s = s_1 s_2 \dots s_n$ una stringa in A di lunghezza n , dove $n \geq p$. Sia $r_1, \dots, r_{n+1}$ la sequenza di stati attraversati da M mentre elabora s , quindi $r_{i+1} = \delta(r_i, s_i)$ per $1 \leq i \leq n$. Questa sequenza ha lunghezza $n + 1$, che è almeno $p + 1$. Due stati tra i primi $p + 1$ elementi nella sequenza devono essere lo stesso stato, per il principio della piccioni e casella. Chiamiamo il primo di questi r_j e il secondo r_l . Poiché r_l si presenta tra le prime $p + 1$ posizioni in una sequenza che inizia in r_1 , abbiamo $l \leq p + 1$. Ora sia $x = s_1 \dots s_{j-1}$, $y = s_j \dots s_{l-1}$ e $z = s_l \dots s_n$.

Poiché x porta M da r_1 a r_j , y porta M da r_j a r_j e z porta M da r_j a r_{n+1} , che è uno stato accettante, M deve accettare $xy^i z$ per $i \geq 0$. Sappiamo che $j \neq l$, perciò $|y| > 0$ e $l \leq p + 1$, perciò $|xy| \leq p$. Quindi tutte le condizioni del pumping lemma sono rispettate. $\square$

Per usare il pumping lemma per provare che un linguaggio B non è regolare, in primo luogo si assuma che B sia regolare per ottenere una contraddizione. Poi si usi il pumping lemma per assicurare l'esistenza di una lunghezza del pumping p tale che tutte le stringhe di lunghezza maggiore o uguale a p in B possano essere iterate. In seguito, si trovi una stringa s in B che ha una lunghezza maggiore o uguale a p , ma che non può essere iterata. Infine, si dimostri che s non può essere iterata considerando tutti i modi di dividere s in x, y e z e, per ogni tale divisione, trovando un valore i tale che $xy^i z \notin B$.

Esempio

Sia $B = \{0^n 1^n \mid n \geq 0\}$. Usiamo il pumping lemma per provare che B non è regolare.

Assumiamo che B sia regolare. Sia p la lunghezza del pumping data dal pumping lemma. Scegliamo $s = 0^p 1^p$. Poiché $s \in B$ ed $|s| \geq p$, il pumping lemma assicura che s può essere divisa in tre parti, $s = xyz$, tali che per ogni $i \geq 0$ la stringa $xy^i z \in B$. Consideriamo tre casi per mostrare che questo risultato è impossibile.

1. La stringa y consiste solo di simboli uguali a 0. In questo caso, la stringa $xyyz$ ha più simboli uguali a 0 che simboli uguali a 1 e quindi non è un elemento di B , violando la condizione 1 del pumping lemma.
2. La stringa y consiste solo di simboli uguali a 1. Anche questo caso porta ad una contraddizione.

3. La stringa y consiste solo di simboli uguali a 0 che di simboli uguali a 1. In questo caso, la stringa $xyyz$ può avere lo stesso numero di simboli uguali a 0 e simboli uguali a 1, ma essi non saranno nell'ordine corretto, con qualche 1 prima di qualche 0. Quindi non è un elemento di B , il che è una contraddizione.

Pertanto una contraddizione è inevitabile se assumiamo che B sia regolare, quindi B non è regolare.

Esempio

Sia $C = \{w \mid w \text{ ha lo stesso numero di simboli uguali a 0 e simboli uguali a 1}\}$. Usiamo il pumping lemma per mostrare che C non è regolare.

Assumiamo al contrario che C sia regolare. Sia p la lunghezza del pumping data dal pumping lemma. Sia s la stringa $0^p 1^p$. Poichè $s \in C$ che ha lunghezza maggiore di p , il pumping lemma assicura che s può essere divisa in tre parti, $s = xyz$, tali che per ogni $i \geq 0$ la stringa $xy^i z \in C$. Se prendiamo x e z uguali alla stringa vuota e y uguale alla stringa $0^p 1^p$, allora $xy^i z$ ha sempre lo stesso numero di simboli uguali a 0 e simboli uguali a 1 e quindi è in C .

Qui la condizione 3 del pumping lemma è utile. Essa stabilisce che quando iteriamo s , essa deve essere divisa in modo che $|xy| \leq p$. Questa restrizione sul modo in cui s può essere divisa rende più facile mostrare che la stringa $s = 0^p 1^p$ che abbiamo scelto non può essere iterata. Se $|xy| \leq p$, allora y deve consistere solo di simboli uguali a 0, perciò $xyyz \notin C$. Quindi, s non può essere iterata.

Scegliere la stringa s in questo esempio richiede più attenzione. Se avessimo scelto $s = (01)^p$, avremmo avuto dei problemi perchè abbiamo bisogno di una stringa che **non può essere iterata** e quella stringa può essere iterata, perfino prendendo in considerazione la condizione 3.

Un metodo alternativo per provare che C non è regolare segue dal fatto che sappiamo che B non è regolare. Se C fosse regolare, anche $C \cap 0^* 1^*$ sarebbe regolare. Questo perchè il linguaggio $0^* 1^*$ è regolare e la classe dei linguaggi regolari è chiusa rispetto all'intersezione.

Capitolo 2

Linguaggi context-free

2.1 Grammatiche context-free

Un esempio di grammatica context-free, che chiamiamo G_1 , è il seguente:

$$A \rightarrow 0A1$$

$$B \rightarrow B$$

$$B \rightarrow \#$$

Una grammatica consiste di un insieme di *regole di sostituzione*, anche chiamate *produzioni*. Tutte le stringhe generate da una grammatica costituiscono il *linguaggio della grammatica*. Denoteremo con $L(G_1)$ il linguaggio della grammatica G_1 .

Ogni linguaggio che può essere generato da una grammatica context-free è chiamato un *linguaggio context-free* (CFL).

DEFINIZIONE 2.1: GRAMMATICA CONTEXT-FREE

Una *grammatica context-free* è una quadrupla (V, Σ, R, S) , dove

1. V è un insieme finito i cui elementi sono chiamati *variabili*,
2. Σ è un insieme finito, disgiunto da V , i cui elementi sono chiamati *terminali*,
3. R è un insieme finito di *regole*, dove ciascuna regola è una variabile e una stringa di variabili e terminali,
4. $S \in V$ è la variabile iniziale.

Se u, v e w sono stringhe di variabili e terminali e $A \rightarrow w$ è una regola della grammatica, diciamo che uAv *produce* uwv , e lo denotiamo con $uAv \Rightarrow uwv$. Diciamo che u *deriva* v , e lo denotiamo con $u \Rightarrow^* v$, se $u = v$ o se esiste una sequenza $u_1, u_2, \dots, u_k$ con $k \geq 0$ e

$$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_k \Rightarrow v.$$

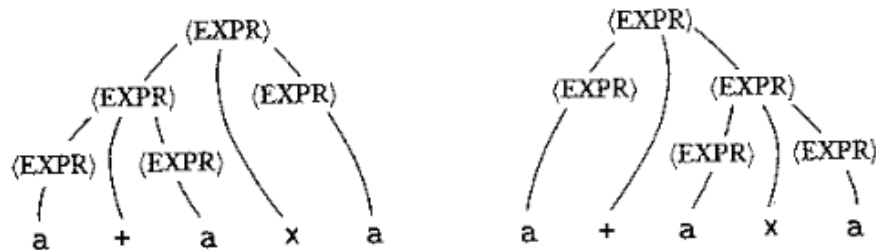
Il *linguaggio della grammatica* è $\{w \in \Sigma^* \mid S \Rightarrow^* w\}$

2.1.1 Ambiguità

Qualche volta una grammatica può generare la stessa stringa in più modi diversi. Una tale stringa avrà diversi alberi sintattici e quindi diversi significati. Questo risultato può essere indesiderabile per alcune applicazioni, come i linguaggi di programmazione, dove un programma dovrebbe avere un'unica interpretazione. Se una grammatica genera la stessa stringa in più modi diversi, diciamo che la stringa è derivata *ambiguamente* in quella grammatica. Se una grammatica genera alcune stringhe ambiguamente, diciamo che la grammatica è *ambigua*. Per esempio, consideriamo la grammatica G_5 :

$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a$$

Questa grammatica genera la stringa $a + a \times a$ ambiguamente.



Questa grammatica non tiene conto delle usuai regole di precedenza e quindi può raggruppare $+$ prima di $\times$ o viceversa. Al contrario la seguente grammatica G_4 genera esattamente lo stesso linguaggio, ma ogni stringa generata ha un solo albero sintattico. $G_4 = (V, \Sigma, R, \langle \text{EXPR} \rangle)$ ove $V = \{\langle \text{EXPR} \rangle, \langle \text{TERM} \rangle, \langle \text{FACTOR} \rangle\}$

e $\Sigma = \{a, +, \times, (,)\}$. Le regole sono

$$\begin{aligned}\langle \text{EXPR} \rangle &\rightarrow \langle \text{EXPR} \rangle + \langle \text{TERM} \rangle \mid \langle \text{TERM} \rangle \\ \langle \text{TERM} \rangle &\rightarrow \langle \text{TERM} \rangle \times \langle \text{FACTOR} \rangle \mid \langle \text{FACTOR} \rangle \\ \langle \text{FACTOR} \rangle &\rightarrow (\langle \text{EXPR} \rangle) \mid a\end{aligned}$$

Quando diciamo che una grammatica genera ambigualmente una stringa, intendiamo che la stringa ha due diversi alberi sintattici, non due differenti derivazioni. Due derivazioni possono differire solo nell'ordine in cui esse sostituiscono le variabili ma non nella loro struttura complessiva. Una derivazione di una stringa w in una grammatica G è una *derivazione a sinistra* se a ogni passo la variabile sostituita è quella che si trova più a sinistra.

DEFINIZIONE 2.2: GRAMMATICA AMBIGUA

Una stringa w è derivata *ambiguamente* in una grammatica context-free G se essa ha due o più diverse derivazioni a sinistra. Una grammatica G è *ambigua* se essa genera qualche stringa ambigualmente.

Quando abbiamo una grammatica ambigua, possiamo trovare una grammatica non ambigua che genera lo stesso linguaggio. Tuttavia, alcuni linguaggi context-free possono essere generati solo da grammatiche ambigue. Tali linguaggi sono chiamati *inerentemente ambigui*.

2.1.2 Forma normale di Chomsky

DEFINIZIONE 2.3: FORMA NORMALE DI CHOMSKY

Una grammatica context-free è in *forma normale di Chomsky* se ogni regola è della forma

$$\begin{aligned}A &\rightarrow BC \\ A &\rightarrow a\end{aligned}$$

dove a è un terminale e A, B e C sono variabili qualsiasi tanne che B e C non possono essere la variabile iniziale.

Inoltre, permettiamo la regola $S \rightarrow \epsilon$, dove S è la variabile iniziale.

Teorema 2.1

Ogni linguaggio context-free è generato da una grammatica context-free in forma normale di Chomsky.

Possiamo trasformare ogni grammatica G in forma normale di Chomsky. La trasformazione ha diversi passi nei quali le regole che violano le condizioni sono rimpiazzate con regole equivalenti che sono adeguate. Innanzitutto, aggiungiamo una nuova variabile iniziale. Poi, eliminiamo tutte le ϵ -regole della forma $A \rightarrow \epsilon$. Eliminiamo anche tutte le *regole unitarie* della forma $A \rightarrow B$. In entrambi i casi, modifichiamo la grammatica in modo da essere sicuri che generi ancora lo stesso linguaggio. Infine, trasformiamo le restanti regole nella forma adeguata.

Dimostrazione. In primo luogo, aggiungiamo una nuova variabile S_0 e la regola $S_0 \rightarrow S$, dove S era la variabile iniziale di partenza. Questo cambiamento garantisce che la variabile iniziale non compare sul lato destro di una regola. In secondo luogo, ci occupiamo di tutte le ϵ -regole. Eliminiamo una ϵ -regola $A \rightarrow \epsilon$, dove A non è la variabile iniziale. Poi, per ogni occorrenza di A sul lato destro di una regola, aggiungiamo una nuova regola con quell'occorrenza cancellata. In altre parole, se $R \rightarrow uAv$ è una regola in cui u e v sono stringhe di variabili e terminali, aggiungiamo la regola $R \rightarrow uv$. Facciamo questo per ogni occorrenza di A , in modo che la regola $R \rightarrow uAvAw$ faccia sì che vengano aggiunte $R \rightarrow uvAw$, $R \rightarrow uAvw$ ed $R \rightarrow uvw$. Se abbiamo la regola $R \rightarrow A$, aggiungiamo $R \rightarrow \epsilon$ a meno che non avevamo precedentemente rimosso la regola $R \rightarrow \epsilon$.

Inoltre, ci occupiamo delle regole unitarie. Eliminiamo una regola unitaria $A \rightarrow B$. Poi, per ogni regola $B \rightarrow u$, aggiungiamo la regola $A \rightarrow u$ a meno che questa non sia una regola unitaria precedentemente cancellata.

Infine, trasformiamo tutte le restanti regole nella forma appropriata. Rimpiazziamo ogni regola $A \rightarrow u_1u_2 \dots u_k$, dove $k \geq 3$ e ciascun u_i è una variabile o un simbolo terminale, con le regole $A \rightarrow u_1A_1$, $A_1 \rightarrow u_2A_2$, $A_2 \rightarrow u_3A_3, \dots$ e $A_{k-2} \rightarrow u_{k-1}u_k$. Le A_i sono nuove variabili. Rimpiazziamo ogni terminale u_i nelle precedenti regole con la nuova variabile U_i e aggiungiamo la regola $U_i \rightarrow u_i$. $\square$

Esempio

Consideriamo la seguente CFG G_6 e trasformiamola in forma normale di Chomsky usando la procedura di conversione appena fornita.

1. La CFG iniziale G_6 è mostrata di seguito. Il risultato dell'applicazione

del primo passo è mostrato sotto.

$$\begin{array}{l} S \rightarrow ASA \mid aB \\ A \rightarrow B \mid S \\ B \rightarrow b \mid \epsilon \end{array}$$

che diventa

$$\begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \\ A \rightarrow B \mid S \\ B \rightarrow b \mid \epsilon \end{array}$$

2. Eliminiamo le ϵ -regole $B \rightarrow \epsilon$.

$$\begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \mid a \\ A \rightarrow B \mid S \mid \epsilon \\ B \rightarrow b \mid \epsilon \end{array}$$

Eliminando la regola $A \rightarrow \epsilon$ otteniamo:

$$\begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S \\ A \rightarrow B \mid S \mid \epsilon \\ B \rightarrow b \end{array}$$

3. Eliminiamo la regola unitaria $S \rightarrow S$.

$$\begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ A \rightarrow B \mid S \mid \epsilon \\ B \rightarrow b \end{array}$$

Eliminiamo la regola unitaria $S_0 \rightarrow S$.

$$\begin{array}{l} S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ A \rightarrow B \mid S \mid \epsilon \\ B \rightarrow b \end{array}$$

Eliminiamo le regole unitarie $A \rightarrow B$ e $A \rightarrow S$.

$$\begin{aligned} S_0 &\rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ S &\rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ A &\rightarrow b \mid ASA \mid aB \mid a \mid SA \mid AS \\ B &\rightarrow b \end{aligned}$$

4. Trasforiamo le rimanenti regole nella forma corretta, aggiungendo variabili e regole supplementari.

$$\begin{aligned} S_0 &\rightarrow AA_1 \mid UB \mid a \mid SA \mid AS \\ S &\rightarrow AA_1 \mid UB \mid a \mid SA \mid AS \\ A &\rightarrow b \mid AA_1 \mid UB \mid a \mid SA \mid AS \\ A_1 &\rightarrow SA \\ U &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

La CNF è utile per risolvere alcuni problemi dei linguaggi context-free, ad esempio il cosiddetto *membership problem* ed è anche utile per provare che alcuni linguaggi sono decidibili.

2.1.3 Membership problem

Il membership problem viene risolto mediante l'algoritmo CYK che permette di stabilire se una stringa w appartiene ad una data grammatica $G = (V, \Sigma, P, S)$. L'algoritmo CYK è un algoritmo di programmazione dinamica in quanto la soluzione al problema $[i, j]$ è costruita dalla combinazione della soluzione al problema $[i, k]$ e la soluzione al problema $[k, j]$.

La grammatica deve essere in CNF in quanto ogni problema è suddiviso in due sottoproblemi e non di più, per limitare la complessità.

Viene usata una tecnica chiamata "*table-filling algorithm*". Viene costruita una tabella triangolare ove per ogni $X_{i,j}$, i rappresenta la lunghezza della stringa e j l'indice di partenza.

$x_{5,1}$				
$x_{4,1}$	$x_{4,2}$			
$x_{3,1}$	$x_{3,2}$	$x_{3,3}$		
$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$	
$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$
w_1	w_2	w_3	w_4	w_5

Esempio

Sia G la grammatica context-free in CNF con le seguenti regole:

$$S \rightarrow AB \mid BC$$

$$A \rightarrow BA \mid a$$

$$B \rightarrow CC \mid b$$

$$C \rightarrow AB \mid a$$

e sia $w = baaba$.

$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$
b	a	a	b	a

Adesso troviamo:

$$X_{2,1} = (X_{1,1}, X_{1,2}) \rightarrow \{B\}\{A, C\} = \{BA, BC\}$$

$$X_{2,2} = (X_{1,2}, X_{1,3}) \rightarrow \{A, C\}\{A, C\} = \{AA, AC, CA, CC\}$$

$$X_{2,3} = (X_{1,3}, X_{1,4}) \rightarrow \{A, C\}\{B\} = \{AB, CB\}$$

$$X_{2,4} = (X_{1,4}, X_{1,5}) \rightarrow \{B\}\{A, C\} = \{BA, BC\}$$

Quindi, dobbiamo trovare le regole che producono BA e BC , AA , AC , CA , CC e così via.

$\{S, A\}$	$\{B\}$	$\{S, C\}$	$\{S, A\}$	
$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$
b	a	a	b	a

Adesso troviamo:

$$X_{3,1} = (X_{1,1}, X_{2,2}), (X_{2,1}, X_{1,3}) \rightarrow \{B\}\{B\} \cup \{S, A\}\{A, C\} = \{BB, SA, SC, AA, AC\}$$

$$X_{3,2} = (X_{1,2}, X_{2,3}), (X_{2,2}, X_{1,4}) \rightarrow \{A, C\}\{S, C\} \cup \{B\}\{B\} = \{AS, AC, CS, CC, BB\}$$

$$X_{3,3} = (X_{1,3}, X_{2,4}), (X_{2,3}, X_{1,5}) \rightarrow \{A, C\}\{S, A\} \cup \{S, C\}\{A, C\} = \{AS, AA, CS, CA, SA, SC, C$$

Quindi, dobbiamo trovare le regole che producono BB, SA, SC, AA e così via.

$\emptyset$	$\{B\}$	$\{B\}$		
$\{S, A\}$	$\{B\}$	$\{S, C\}$	$\{S, A\}$	
$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$
b	a	a	b	a

Adesso troviamo:

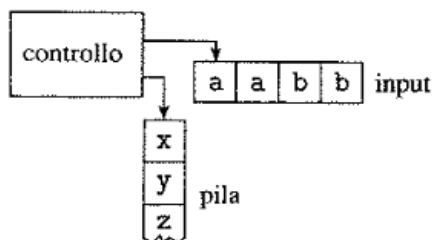
$$X_{4,1} = (X_{1,1}, X_{3,2}), (X_{2,1}, X_{2,3}), (X_{3,1}, X_{1,4}) = \{BB, SS, SC, AS, AC, B\}$$

$$X_{4,2} = (X_{1,2}, X_{3,3}), (X_{2,2}, X_{2,4}), (X_{3,2}, X_{1,5}) = \{AB, CB, BS, BA, BC\}$$

$\{S, A, C\}$				
$\emptyset$	$\{S, A, C\}$			
$\emptyset$	$\{B\}$	$\{B\}$		
$\{S, A\}$	$\{B\}$	$\{S, C\}$	$\{S, A\}$	
$\{B\}$	$\{A, C\}$	$\{A, C\}$	$\{B\}$	$\{A, C\}$
b	a	a	b	a

2.2 Automi a pila

Gli *automi a pila* (*pushdown automata*) sono come gli automi finiti non deterministici ma hanno una componente in più chiamata *pila* (*stack*). La pila fornisce memoria aggiuntiva oltre alla quantità finita di essa disponibile nel controllo. La pila consente a tali automi di riconoscere alcuni linguaggi non regolari. Gli automi a pila sono computazionalmente equivalenti alle grammatiche context-free. Questa equivalenza è utile poichè essa ci dà due alternative per provare che un linguaggio è context-free. La figura seguente è una rappresentazione schematica di un automa finito. Il controllo rappresenta gli stati e la funzione di transizione, il nastro contiene la stringa di input, e la freccia rappresenta la testina sull'input, che indica il successivo simbolo di input da leggere.



Un PDA può scrivere simboli nella pila e rileggerli in seguito. Scrivere un simbolo “spinge giù” tutti gli altri simboli nella pila. In un qualunque momento il simbolo sulla cima (*top*) della pila può essere letto e rimosso. I rimanenti simboli allora ritornano più in alto. L’operazione di scrivere un simbolo dalla pila è spesso chiamata *push*, quella di eliminare un simbolo dalla pila è spesso chiamata *pop*. Una pila è molto importante perchè può mantenere una quantità non limitata di informazioni. Ricordiamo che un automa finito non può riconoscere il linguaggio $\{0^n 1^n \mid n \geq 0\}$ poichè non può memorizzare numeri molto grandi nella sua memoria finita. Un PDA è in grado di riconoscere questo linguaggio poichè può usare la sua pila per memorizzare il numero di simboli uguali a 0 che ha visto. Gli automi a pila possono essere non deterministici. Automi a pila non deterministici e deterministici non sono computazionalmente equivalenti. Gli automi a pila non deterministici riconoscono alcuni linguaggi che nessun automa a pila deterministico può riconoscere. Gli automi a pila non deterministici sono computazionalmente equivalenti alle grammatiche context-free.

2.2.1 Definizione formale di automa a pila

La definizione di automa a pila è simile a quella di un automa finito, tranne che per la pila. La macchina può usare differenti alfabeti per il suo input e la sua pila, quindi ora specifichiamo sia un alfabeto di input Σ sia un alfabeto di pila Γ .

Nel cuore di ogni definizione di automa c'è la funzione di transizione, che descrive il suo comportamento. Ricordiamo che $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ e $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$. Il dominio della funzione di transizione è $Q \times \Sigma_\epsilon \times \Gamma_\epsilon$. Quindi lo stato corrente, il prossimo simbolo di input letto, e il simbolo sulla cima della pila determinano la mossa seguente di un automa a pila. L'uno o l'altro simbolo può essere ϵ , il che determina che la macchina si muove senza leggere un simbolo di input o senza leggere un simbolo dalla pila.

Per quanto riguarda il codominio della funzione di transizione, dobbiamo considerare cosa può fare l'automata quando è in una specifica situazione. Esso può trovarsi in un nuovo stato ed eventualmente scrivere un simbolo sulla cima della pila. La funzione δ può indicare quest'azione restituendo un elemento di Q insieme a un elemento di Γ_ϵ , cioè un elemento di $Q \times \Gamma_\epsilon$. Poiché permettiamo il non determinismo in questo modello, una situazione può avere diverse mosse successive lecite. La funzione di transizione ingloba il non determinismo nel modo usuale, restituendo un insieme di elementi di $Q \times \Gamma_\epsilon$, cioè un elemento di $\mathcal{P}(Q \times \Gamma_\epsilon)$. Quindi la funzione di transizione δ assume la forma: $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$

DEFINIZIONE 2.4: AUTOMA A PILA

Un *automa a pila* è una sestupla $(Q, \Sigma, \Gamma, \delta, q_0, F)$ dove, Q, Σ, Γ ed F sono tutti insiemi finiti, e

1. Q è l'insieme degli stati,
2. Σ è l'alfabeto dell'input,
3. Γ è l'alfabeto della pila,
4. $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$ è la funzione di transizione,
5. $q_0 \in Q$ è lo stato iniziale,
6. $F \subseteq Q$ è l'insieme degli stati accettanti.

Un automa a pila $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ computa nel seguente modo. Accetta un input w se w può essere scritto come $w = w_1 w_2 \dots w_m$ dove ciascun $w_i \in \Sigma_\epsilon$ ed esistono sequenze di stati $r_0, r_1, \dots, r_m \in Q$ e di stringhe $s_0, s_1, \dots, s_m \in \Gamma^*$

che soddisfano le tre condizioni seguenti. Le stringhe s_i rappresentano la sequenza del contenuto della pila che M ha su un ramo accettante della computazione.

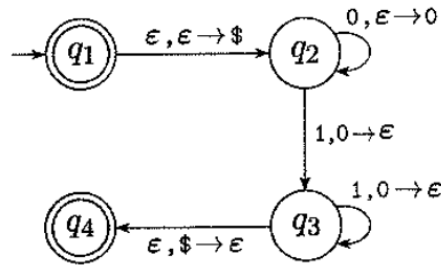
1. $r_0 = q_0$ e $s_0 = \epsilon$. Questa condizione significa che M inizia correttamente, nello stato iniziale e con una pila vuota.
2. Per $i = 0, \dots, m - 1$ abbiamo $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$ dove $s_i = at$ ed $s_{i+1} = bt$ per qualche $a, b \in \Gamma_\epsilon$ e $t \in \Gamma^*$. Questa condizione afferma che M si muove correttamente in base allo stato, al simbolo di pila, e al prossimo simbolo in input.
3. $r_m \in F$. Questa condizione afferma che alla fine dell'input M si trova in uno stato accettante.

Esempio

Quello che segue è la descrizione formale del PDA che riconosce il linguaggio $\{0^n 1^n \mid n \geq 0\}$. Sia M_1 la sestupla $(Q, \Sigma, \Gamma, \delta, q_1, F)$, dove

- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, \$\}$
- $F = \{q_1, q_4\}$

L'automa è il seguente.



La definizione di un PDA non contiene alcun meccanismo esplicito per permettere al PDA di controllare se la pila è vuota. Questo PDA è in grado di ottenere

lo stesso effetto inserendo inizialmente un simbolo speciale \$ nella pila. Allora, nel caso in cui veda il \$ di nuovo, sa che la pila è di fatto vuota.

Analogamente, i PDA non possono controllare esplicitamente se hanno raggiunto la fine della stringa di input. Questo PDA è in grado di ottenere lo stesso risultato poichè lo stato accettante potrebbe essere eventualmente raggiunto solo quando la macchina è alla fine dell'input. Quindi possiamo assumere che i PDA possono controllare la fine dell'input e sappiamo che possiamo implementarlo nello stesso modo.

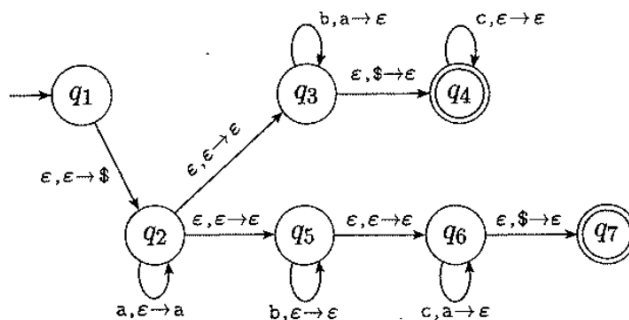
Esempio

Questo esempio illustra un PDA che riconosce il linguaggio:

$$\{a^i b^j c^k \mid i, j, k \geq 0 \text{ e } i = j \text{ oppure } i = k\}$$

Informalmente, il PDA per questo linguaggio opera dapprima leggendo e inserendo le a nella pila. Quando le a sono terminate, la macchina le ha tutte nella pila, quindi può abbinarle con le b o con le c . Quest'operazione è complicata in quanto la macchina non sa se far corrispondere le a con le b o con le c . In questo caso il non determinismo risulta utile.

Usando il non determinismo, il PDA può ipotizzare di far corrispondere le a con le b o con le c . Il PDA è il seguente.

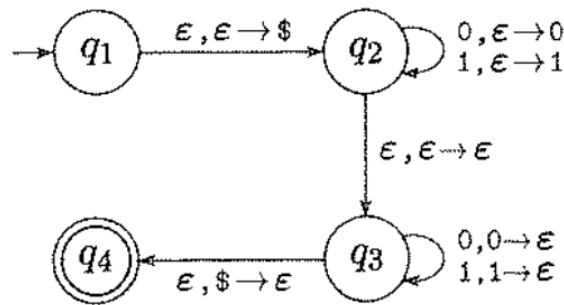


Esempio

In questo esempio diamo un PDA M_3 che riconosce il linguaggio $\{ww^R \mid w \in \{0, 1\}^*\}$.

Il PDA inizia inserendo nella pila i simboli letti. In qualunque momento ipotizza non deterministicamente di aver raggiunto la prima metà della stringa

e quindi cambia azione, eliminando dalla pila un simbolo per ciascun simbolo letto, se essi coincidono. Se essi coincidono sempre e la pila si svuota nello stesso momento in cui l'input è terminato, accetta; altrimenti rifiuta.



2.2.2 Equivalenza con le grammatiche context-free

Le grammatiche context-free e i PDA sono computazionalmente equivalenti e entrambi sono in grado di descrivere la classe dei linguaggi context-free.

Teorema 2.2: Equivalenza delle CFG con i PDA

Un linguaggio è context free se e solo se esiste un automa a pila che lo riconosce.

Per i teoremi “se e solo se”, abbiamo due enunciati da provare, nelle due direzioni.

LEMMA 2.1

Se un linguaggio è context-free, allora esiste un automa a pila che lo riconosce.

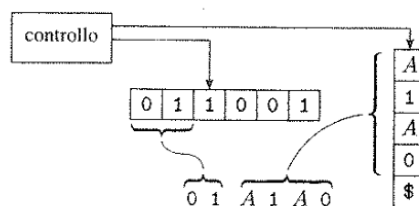
Sia A un CFL. Dalla definizione sappiamo che esiste una CFG, G , che genera A . Mostriamo come trasformare G in un PDA equivalente, che chiamiamo P .

Il PDA P che ora descriviamo, opererà accettando il suo input w , se G genera tale input, determinando se esiste una derivazione per w . Noi progettiamo P in modo che possa stabilire una serie di sostituzioni che usano le regole di G possa condurre dalla variabile iniziale a w .

Una delle difficoltà nell'esaminare se esiste una derivazione per w consiste nel capire quali sostituzioni fare. Il non determinismo di un PDA gli consente di ipotizzare la sequenza di sostituzioni giuste. In ogni passo della derivazione, una

delle regole per una particolare variabile è scelta non deterministicamente e usata per sostituire quella variabile. Il PDA P inizia scrivendo la variabile iniziale sulla sua pila. Esso passa attraverso una serie di stringhe intermedie, facendo una sostituzione dietro l'altra. Infine può giungere a una stringa che contiene solo simboli terminali, il che significa che usato la grammatica per derivare una stringa. Allora P accetta se questa stringa è identica alla stringa che ha ricevuto in input.

Implementare questa strategia su un PDA richiede un'idea supplementare. Dobbiamo capire come il PDA immagazzina le stringhe intermedie quando passa dall'una all'altra. Usare semplicemente la pila per immagazzinare ciascuna stringa intermedia è allettante. Tuttavia non funziona del tutto poiché il PDA ha bisogno di trovare le variabili nelle stringhe intermedie e fare sostituzioni. Il PDA può accedere solo al simbolo sulla cima della pila e questo potrebbe essere un simbolo terminale e non una variabile. Il modo per aggirare questo problema è mantenere solo parte della stringa intermedia sulla pila: i simboli che iniziano con la prima variabile nella stringa intermedia. Tutti i simboli terminali che compaiono prima della prima variabile sono subito abbinati con i simboli nella stringa di input. La figura seguente mostra il PDA P .



Quello che segue è una descrizione informale di P .

1. Inserisce il simbolo marcatore $\$$ e la variabile iniziale sulla pila.
2. Ripete i seguenti passi.
 - (a) Se sulla cima della pila c'è il simbolo di una variabile A , sceglie non deterministicamente una delle regole per A e sostituisce A con la stringa sul lato destro della regola.
 - (b) Se sulla cima della pila c'è un simbolo terminale a , legge il simbolo seguente dall'input e lo confronta con a . Se essi sono uguali, ripete. Se essi non sono uguali, rifiuta su questo ramo del non determinismo.

- (c) Se sulla cima della pila c'è il simbolo $\$,$ entra nello stato accettante. In questo modo accetta l'input se esso è stato completamente letto.

Dimostrazione. Ora diamo i dettagli formali della costruzione del PDA $P = (Q, \Sigma, \Gamma, \delta, q_{start}, F)$. Per rendere più chiara la costruzione, usiamo una notazione abbreviata per la funzione di transizione. Questa notazione fornisce un modo per scrivere un'intera stringa sulla pila in un passo della macchina. Possiamo simulare quest'azione introducendo stati aggiuntivi per scrivere la stringa un simbolo alla volta.

Siano q ed r stati del PDA e siano $a \in \Sigma_\epsilon$ ed $s \in \Gamma_\epsilon$. Supponiamo di volere che il PDA vada da q ad r quando legge a ed elimina s . Inoltre, vogliamo che esso inserisca l'intera stringa $u = u_1 \cdots u_l$ sulla pila simultaneamente. Possiamo eseguire quest'azione introducendo nuovi stati $q_1, \dots, q_{l-1}$ e definendo la funzione di transizione come segue:

$$\begin{aligned} \delta(q, a, s) &\text{ contiene } (q_1, u_l), \\ \delta(q_1, \epsilon, \epsilon) &= \{(q_2, u_{l-1})\}, \\ \delta(q_2, \epsilon, \epsilon) &= \{(q_3, u_{l-2})\}, \\ &\vdots \\ \delta(q_{l-1}, \epsilon, \epsilon) &= \{(r, u_1)\} \end{aligned}$$

Useremo la notazione $(r, u) \in \delta(q, a, s)$ per denotare che quando q è lo stato in cui si trova l'automa, a è il prossimo simbolo di input ed s è il simbolo sulla cima della pila, il PDA può leggere a ed eliminare s , poi inserire la stringa u nella pila e passare nello stato r . La figura seguente ne mostra la realizzazione.

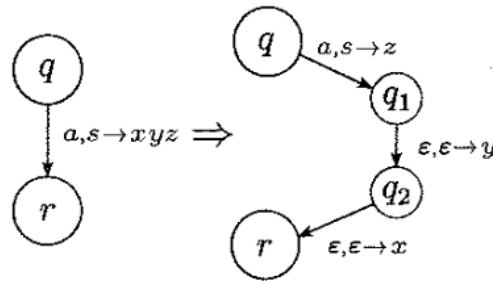


Figura 2.1: Implementazione dell'abbreviazione $(r, xyz) \in \delta(q, a, s)$

Gli stati di P sono $Q = \{q_{start}, q_{loop}, q_{accept}\} \cup E$, dove E è l'insieme degli stati necessari per realizzare l'abbreviazione appena descritta. Lo stato iniziale è q_{start} . L'unico stato accettante è q_{accept} .

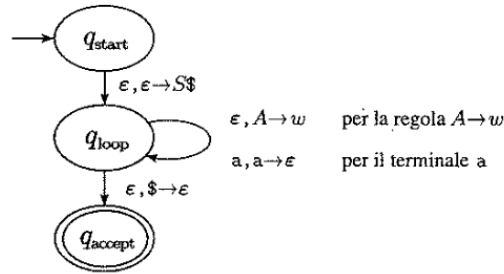
La funzione di transizione è definita come segue. Cominciamo inizializzando la pila inserendo i simboli $\$$ e S , realizzando così il passo 1 nella descrizione informale: $\delta(q_{start}, \epsilon, \epsilon) = \{(q_{loop}, S\$)\}$. Poi aggiungiamo le transizioni per il ciclo principale del passo 2.

In primo luogo, trattiamo il caso (a) in cui la cima della pila contiene una variabile. Poniamo $\delta(q_{loop}, \epsilon, A) = \{(q_{loop}, w) \mid \text{dove } A \rightarrow w \text{ è una regola in } R\}$.

In secondo luogo, trattiamo il caso (b) in cui la cima della pila contiene un terminale. Poniamo $\delta(q_{loop}, a, a) = \{(q_{loop}, \epsilon)\}$.

Infine, trattiamo il caso (c) in cui il marcatore scelto per indicare la pila vuota $\$$ è sulla cima della pila. Poniamo $\delta(q_{loop}, \epsilon, \$) = \{(q_{accept}, \epsilon)\}$.

Il diagramma di stato è mostrato nella seguente figura.



□

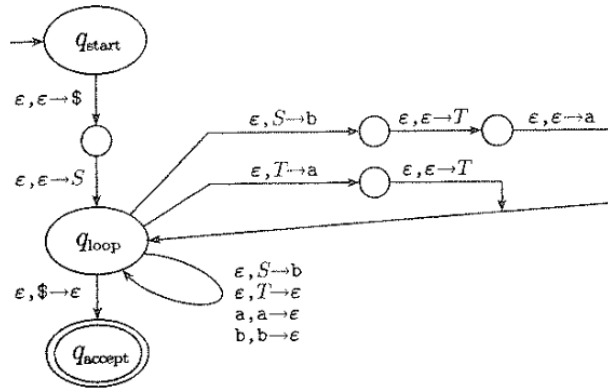
Esempio

Costruiamo un PDA P_1 dalla seguente CFG G .

$$S \rightarrow aTb \mid b$$

$$T \rightarrow Ta \mid \epsilon$$

La funzione di transizione è mostrata nel diagramma seguente.



Ora dimostriamo la direzione inversa del teorema **equivalenza delle CFG con i PDA**. Per la direzione diretta, abbiamo dato una procedura per trasformare una CFG in PDA. Ora vogliamo dare una procedura per andare nella direzione opposta: trasformare un PDA in una CFG. Progettiamo la grammatica per simulare l'automa. Questo compito è impegnativo perchè “programmare” un automa è più facile che “programmare” una grammatica.

LEMMA 2.2

Se un linguaggio è riconosciuto da un automa a pila, allora esso è context-free.

Abbiamo un PDA P e vogliamo costruire una CFG G che genera tutte le stringhe che P accetta. In altre parole, G dovrebbe generare una stringa se quella stringa fa andare il PDA dal suo stato iniziale a uno stato accettante.

Per raggiungere questo risultato, progettiamo una grammatica che fa un pò in più. Per ciascuna coppia di stati p e q in P , la grammatica avrà una variabile A_{pq} . Questa variabile genera tutte le stringhe che possano portare P da p con pila vuota a q con pila vuota. Si osservi che tale stringhe possono anche condurre P da p a q , indipendentemente dal contenuto della pila in p , lasciando la pila in q nella stessa condizione in cui era in p . Innanzitutto, semplifichiamo il nostro compito modificando leggermente P per munirlo delle seguenti tre caratteristiche.

1. Ha un unico stato accettante, q_{accept} .
2. Svuota la sua pila prima di accettare.

3. Ciascuna transizione inserisce un simbolo sulla pila o ne elimina uno dalla pila, ma non fa entrambe le azioni contemporaneamente.

Dare a P le caratteristiche 1 e 2 è facile. Per munirlo della caratteristica 3, sostituiamo ciascun transizione che contemporaneamente elimina ed inserisce simboli con una sequenza di due transizioni che attraversa un nuovo stato, e sostituiamo ogni transizione che non elimina nè inserisce simboli con una sequenza di due transizioni che inserisce e poi elimina un simbolo arbitrario della pila.

Per progettare G in modo che A_{pq} generi tutte le stringhe che portano P da p a q , iniziando e terminando con una pila vuota, dobbiamo capire come P agisce su queste stringhe. Per ognuna di tali stringhe x , la prima mossa di P su x deve essere un push, poichè ogni mossa è un push o un pop e P non può eliminare da una pila vuota. Analogamente, l'ultima mossa su x deve essere un pop perchè alla fine la pila deve essere vuota.

Durante la computazione di P su x si presentano due eventualità. O il simbolo eliminato alla fine è il simbolo che era stato inserito all'inizio, oppure no. Nel primo caso, la pila potrebbe essere vuota solo all'inizio e alla fine della computazione di P su x . Altrimenti, il simbolo inizialmente inserito deve essere stato eliminato in qualche punto prima della fine di x e quindi la pila si svuota in questo punto. Simuliamo la prima possibilità con la regola $A_{pq} \rightarrow aA_{rs}b$, dove a è l'input letto nella prima mossa, b è l'input letto nell'ultima mossa, r è lo stato che segue p ed s è lo stato che precede q . Simuliamo la seconda possibilità con la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ dove r è lo stato in cui la pila diventa vuota.

Dimostrazione. Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{accept}\})$ e costruiamo G . L'insieme delle variabili di G è $\{A_{pq} \mid p, q \in Q\}$. La variabile iniziale è $A_{q_0, q_{accept}}$. Ora descriviamo le regole di G nei seguenti tre punti.

1. $\forall p, q, r, s \in Q, u \in \Gamma$ e $a, b \in \Sigma_\epsilon$, se $\delta(p, a, \epsilon)$ contiene (r, u) e $\delta(s, b, u)$ contiene (q, ϵ) , poni la regola $A_{pq} \rightarrow aA_{rs}b$ in G
2. $\forall p, q, r \in Q$ poni la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ in G
3. Infine, $\forall p \in Q$ poni la regola $A_{pp} \rightarrow \epsilon$ in G .

Ora proviamo che questa costruzione funziona dimostrando che A_{pq} genera x se e solo se x porta P da p con la pila vuota a q con la pila vuota. Consideriamo ogni direzione di "se e solo se" come un enunciato separato.

PROPOSIZIONE 2.3

Se A_{pq} genera x , allora x può portare P da p con pila vuota a q con la pila vuota.

Dimostriamo questo enunciato per induzione sul numero dei passi nella derivazione di x da A_{pq} . **Base:** La derivazione è in un solo passo.

Una derivazione in un solo passo deve usare una regola il cui lato destro non contenga variabili. Le uniche regole in G con nessuna variabile sul lato destro sono $A_{pp} \rightarrow \epsilon$. Ovviamente, l'input ϵ porta P da p con la pila vuota a p con la pila vuota e quindi la base è dimostrata.

Passo induttivo: Assumiamo che l'enunciato sia vero per le derivazioni di lunghezza al più k , dove $k \geq 1$, e proviamo che esso è vero per le derivazioni di lunghezza $k + 1$.

Supponiamo che $A_{pq} \xRightarrow{*} x$ in $k + 1$ passi. Il primo passo in questa derivazione è $A_{pq} \Rightarrow aA_{rs}b$ oppure $A_{pq} \Rightarrow A_{pr}A_{rq}$. Trattiamo questi due casi separatamente.

Nel primo caso, consideriamo la parte y di x che A_{rs} genera, quindi $x = ayb$. Poichè $A_{rs} \xRightarrow{*} y$ in k passi, l'ipotesi induttiva ci dice che P può andare da r con la pila vuota a s con la pila vuota. Poichè $A_{pq} \rightarrow aA_{rs}b$ è una regola di G , $\delta(p, a, \epsilon)$ contiene (r, u) e $\delta(s, b, u)$ contiene (q, ϵ) , per qualche simbolo di pila u . Quindi, se P inizia in p con la pila vuota, dopo aver letto a può andare nello stato r e inserire u in cima alla pila. Poi, la lettura della stringa y può portarlo in s e lasciare u sulla pila. In seguito dopo aver letto b può andare nello stato q ed eliminare u dalla pila. Pertanto, x può portare P da p con la pila vuota a q con la pila vuota.

Nel secondo caso, consideriamo le parti y e z di x che A_{pr} e A_{rq} rispettivamente generano, quindi $x = yz$. Poichè $A_{pr} \xRightarrow{*} y$ in al più k passi e $A_{rq} \xRightarrow{*} z$ in al più k passi, l'ipotesi induttiva ci dice che y può portare P da p a r e z può portare P da r a q , con la pila vuota all'inizio e alla fine della computazione. Quindi x può portare P da p con la pila vuota a q con la pila vuota. Questo completa la prova del passo induttivo.

PROPOSIZIONE 2.4

Se x può portare P da p con la pila vuota a q con la pila vuota, A_{pq} genera x .

Dimostriamo questo enunciato per induzione sul numero dei passi nella computazione di P da p a q con pile vuote sull'input x .

Base: La computazione è in 0 passi.

Se una computazione è in 0 passi, essa inizia e termina nello stesso stato p . Quindi dobbiamo mostrare che $A_{pp} \xRightarrow{*} x$. P non può leggere alcun carattere in 0 passi, quindi $x = \epsilon$. Per costruzione, G ha la regola $A_{pp} \rightarrow \epsilon$, perciò la base è dimostrata.

Passo induttivo: Assumiamo l'enunciato vero per le computazioni di lunghezza al più k , dove $k \geq 0$, e dimostriamo che è vero per le computazioni di lunghezza $k + 1$.

Supponiamo che P abbia una computazione dove x porta da p a q con pile vuote in $k + 1$ passi. O la pila è vuota solo all'inizio e alla fine di questa computazione oppure essa si svuota anche altrove.

Nel primo caso, il simbolo che è stato inserito nella prima mossa deve essere lo stesso simbolo che è stato rimosso dall'ultima mossa. Chiamiamo u questo simbolo. Sia a il simbolo di input letto nella prima mossa, b il simbolo di input letto nell'ultima mossa, r lo stato dopo la prima mossa ed s lo stato prima dell'ultima mossa. Allora $\delta(p, a, \epsilon)$ contiene (r, u) e $\delta(s, b, u)$ contiene (q, ϵ) e quindi la regola $A_{pq} \rightarrow aA_{rs}b \in G$.

Sia y la parte di x senza a e b , quindi $x = ayb$. L'input y può portare P da r a s senza toccare il simbolo u che è sulla cima della pila e quindi P può andare da r con pila vuota a s con pila vuota sull'input y . Abbiamo eliminato il primo e l'ultimo passo dei $k + 1$ passi della computazione iniziale su x , perciò la computazione su y ha $k - 1$ passi. Quindi l'ipotesi induttiva ci dice che $A_{rs} \xRightarrow{*} y$. Allora $A_{pq} \xRightarrow{*} x$.

Nel secondo caso, sia r uno stato in cui la pila si svuota oltre che all'inizio o alla fine della computazione su x . Allora le parti della computazione da p ad r e da r a q contengono al più k passi. Sia y l'input letto nella prima parte e sia z l'input letto nella seconda parte. L'ipotesi induttiva ci dice che $A_{pr} \xRightarrow{*} y$ e $A_{rq} \xRightarrow{*} z$. Poichè la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ è in G , $A_{pq} \xRightarrow{*} x$ e la dimostrazione è completata. $\square$

2.3 Linguaggi non context-free

In questa sezione presentiamo una tecnica per dimostrare che alcuni linguaggi non sono context-free. Precedentemente, per dimostrare che un linguaggio non era regolare, abbiamo utilizzato il pumping lemma. A questo punto presentiamo

un pumping lemma simile per i linguaggi context-free. Esso stabilisce che per ogni linguaggio context-free esiste un particolare valore chiamato la *lunghezza del pumping* tale che tutte le stringhe più lunghe nel linguaggio possono essere iterate. Questa volta il significato di iterazione è più complesso.

2.3.1 Il pumping lemma per i linguaggi context-free

Teorema 2.3: Il pumping lemma per i linguaggi context-free

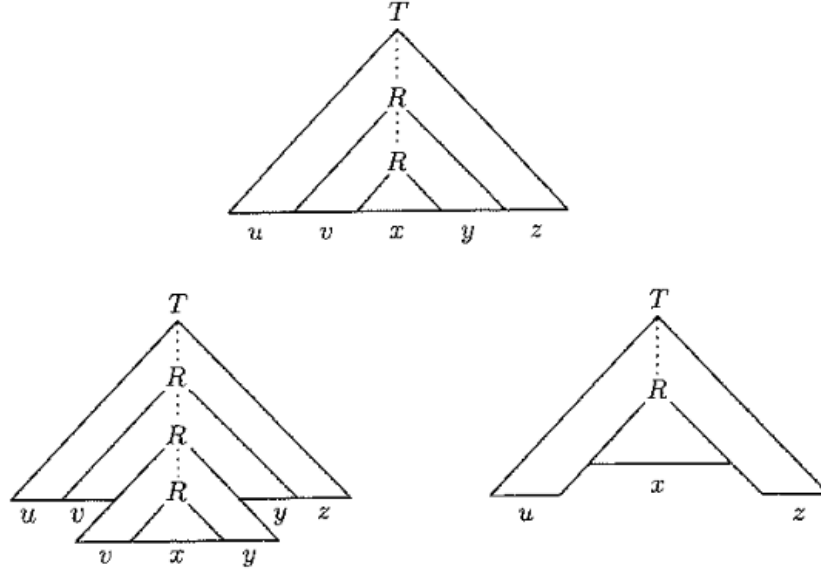
Se A è un linguaggio context-free, allora esiste un numero p (lunghezza del pumping) tale che, se s è qualsiasi stringa in A di lunghezza almeno p , allora s può essere divisa in cinque parti $s = uvxyz$ che soddisfano le condizioni.

1. $\forall i \geq 0, uv^i xy^i z \in A$
2. $|vy| > 0$
3. $|vxy| \leq p$

Quando s è divisa in $uvxyz$, la condizione 2 afferma che v oppure y non è la stringa vuota. Altrimenti il teorema sarebbe banalmente vero. La condizione 3 afferma che le parti v , x e y insieme hanno lunghezza al più p . Questa condizione tecnica a volte è utile per dimostrare che alcuni linguaggi non sono context-free.

Sia A un CFL e sia G una CFG che lo genera. Dobbiamo dimostrare che ogni stringa sufficientemente lunga s in A può essere iterata e restare in A . L'idea dietro questa strategia è semplice.

Sia s una stringa molto lunga in A . Poiché s è in A , essa è derivabile da G e quindi ha un albero sintattico. L'albero sintattico per s deve essere molto alto perchè s è molto lunga. Cioè l'albero sintattico deve contenere un cammino lungo dalla variabile alla radice dell'albero a uno dei simboli terminali su una foglia. Per il principio della piccionaia, qualche simbolo di variabile R si deve ripetere in questo cammino lungo. Come mostra la figura seguente, questa ripetizione ci permette di sostituire il sottoalbero sotto la seconda occorrenza di R con il sottoalbero sotto la prima occorrenza di R e ottenere ancora un albero sintattico consentito. Pertanto, possiamo dividere s in cinque parti $uvxyz$ e possiamo replicare il secondo e il quarto pezzo e ottenere una stringa ancora nel linguaggio. In altre parole, $uv^i xy^i z$ è in A per ogni $i \geq 0$.



Dimostrazione. Sia G una CFG per il CFL A . Sia b il massimo numero di simboli nel lato destro di una regola (assumiamo che sia almeno 2). Sappiamo che, in ogni albero sintattico costruito usando questa grammatica, un nodo non può avere più di b figli. In altre parole, ci sono al più b foglie in un passo dalla variabile iniziale; ci sono al più b^2 foglie in 2 passi dalla variabile iniziale; e ci sono al più b^h foglie in h passi dalla variabile iniziale. Quindi, se l'altezza dell'albero sintattico è al più h , la lunghezza della stringa generata è al più b^h . Viceversa, se una stringa generata ha lunghezza maggiore o uguale a $b^h + 1$, ciascuno dei suoi alberi sintattici deve avere un'altezza maggiore o uguale a $h + 1$.

Sia $|V|$ il numero delle variabili in G . Poniamo p , la lunghezza del pumping, uguale a $b^{|V|+1}$. Ora se s è una stringa in A e la sua lunghezza è maggiore o uguale a p , il suo albero sintattico deve avere altezza maggiore o uguale a $|V| + 1$, poichè $b^{|V|+1} \geq b^{|V|} + 1$.

Per vedere come iterare tale stringa s , sia τ uno dei suoi alberi sintattici. Se s ha diversi alberi sintattici, scegliamo un albero sintattico τ che abbia il più piccolo numero di nodi. Sappiamo che τ deve avere altezza maggiore o uguale a $|V| + 1$, quindi il suo cammino più lungo dalla radice a una foglia ha lunghezza almeno $|V| + 1$. Questo cammino ha almeno $|V| + 2$ nodi; uno etichettato da un terminale, gli altri etichettati da variabili. Quindi questo cammino ha

almeno $|V| + 1$ variabili. Poichè G ha solo $|V|$ variabili, qualche variabile R è presente più di una volta su questo cammino. Per un utilizzo successivo, scegliamo R in modo che una variabile che si ripete tra le $|V| + 1$ variabili più in basso su questo cammino.

Dividiamo s in $uvxyz$. Ogni occorrenza di R ha un sottoalbero sotto essa che genera una parte della stringa s . L'occorrenza più in alto di R ha un sottoalbero più grande e genera vxy , mentre l'occorrenza più in basso genera solo x come sottoalbero più piccolo. Entrambi questi sottoalberi sono generati dalla stessa variabile, quindi possiamo sostituire l'uno con l'altro e ottenere ancora un albero sintattico corretto. Sostituire ripetutamente il più piccolo con il più grande fornisce gli alberi sintattici per le stringhe $uv^i xy^i z$ per ogni $i > 1$. Sostituire il più grande con il più piccolo genera la stringa uxz . Questo dimostra la condizione 1 del pumping lemma. Dimostriamo le condizioni 2 e 3.

Per ottenere la condizione 2, dobbiamo essere sicuri che v e y non sono entrambe ϵ . Se lo fossero, l'albero sintattico ottenuto sostituendo il più piccolo sottoalbero al più grande avrebbe meno nodi di τ e genererebbe ancora s . Questo non è possibile perchè abbiamo scelto τ in modo che sia un albero sintattico per s con il più piccolo numero di nodi. Questa è la ragione per aver selezionato così τ .

Per ottenere la condizione 3, dobbiamo essere sicuri che vxy ha lunghezza al più p . Nell'albero sintattico per s l'occorrenza più in alto di R genera vxy . Abbiamo scelto R in modo che entrambe le occorrenze di essa cadano nelle $|V| + 1$ variabili più in basso del cammino e abbiamo scelto il più lungo cammino nell'albero sintattico, in modo che il sottoalbero in cui R genera vxy sia alto al più $|V| + 1$. Un albero con questa altezza può generare una stringa di lunghezza al più $b^{|V|+1} = p$ $\square$

Esempio

Usiamo il pumping lemma per mostrare che il linguaggio $B = \{a^n b^n c^n \mid n \geq 0\}$ non è context-free.

Assumiamo che B sia un CFL e giungiamo a una contraddizione. Sia p la lunghezza del pumping per B la cui esistenza è garantita dal pumping lemma. Scegliamo la stringa $s = a^p b^p c^p$. Ovviamente s è un elemento di B e di lunghezza almeno p . Il pumping lemma afferma che s può essere iterata, ma noi mostriamo che non può esserlo. In altre parole, mostriamo che, non importa come dividiamo s in $uvxyz$, una delle tre condizioni del pumping lemma è violata.

In primo luogo, la condizione 2 stabilisce che v o y non è vuota. Allora consideriamo due casi, a seconda che le sottostringhe v e y contengano più di un tipo di simbolo dell'alfabeto o no.

1. Quando entrambe v e y contengono solo un tipo di simboli dell'alfabeto, v non contiene entrambi i simboli a e b o entrambi i simboli b e c e lo stesso vale per y . In questo caso, la stringa uv^2xy^2z non può contenere lo stesso numero di a , b e c . Quindi, essa non può essere un elemento di B . Abbiamo raggiunto la contraddizione.
2. Quando v o y contengono più di un tipo di simbolo, uv^2xy^2z può contenere un equal numero dei tre simboli dell'alfabeto ma non nell'ordine corretto. Perciò essa non può essere un elemento di B e si verifica un assurdo.

Uno di questi casi deve verificarsi. Poichè entrambi i casi conducono a un assurdo, la contraddizione è inevitabile. Quindi l'assunzione che B sia un CFL deve essere falsa. Pertanto abbiamo provato che B non è CFL.